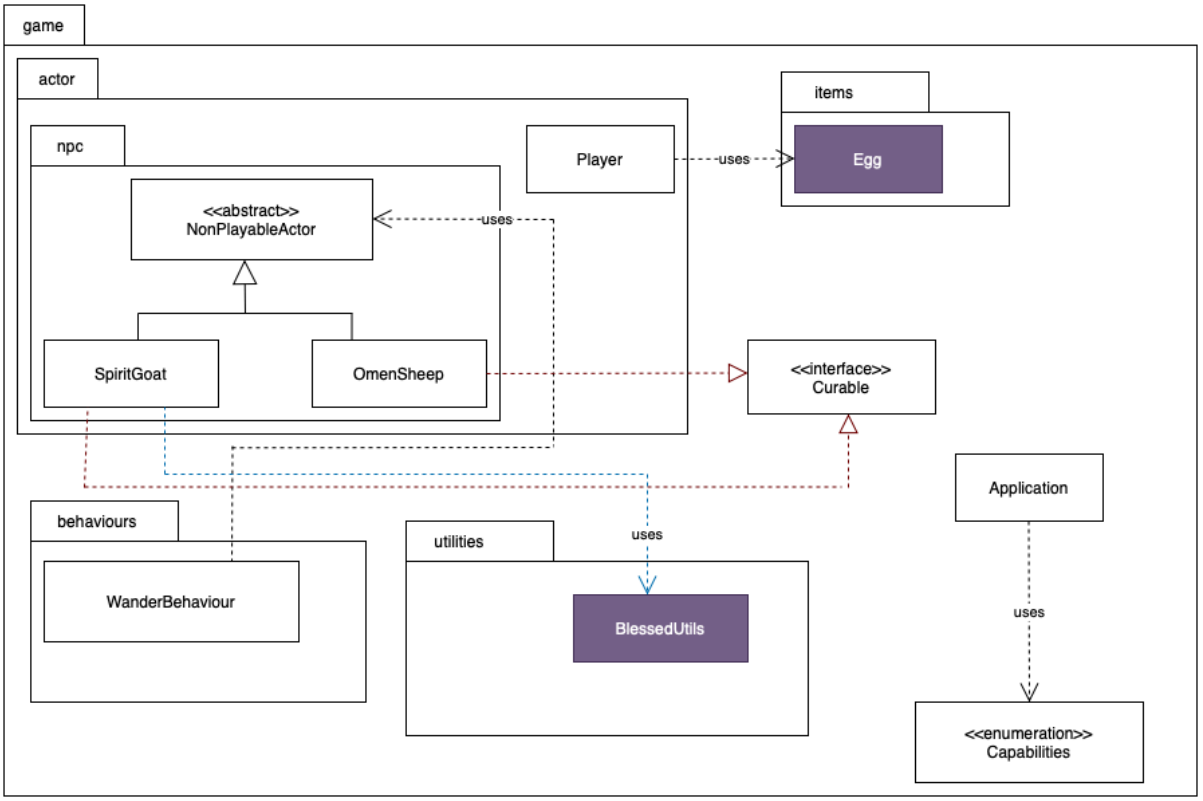
# FIT2099 – Assignment 2 – Design Rational

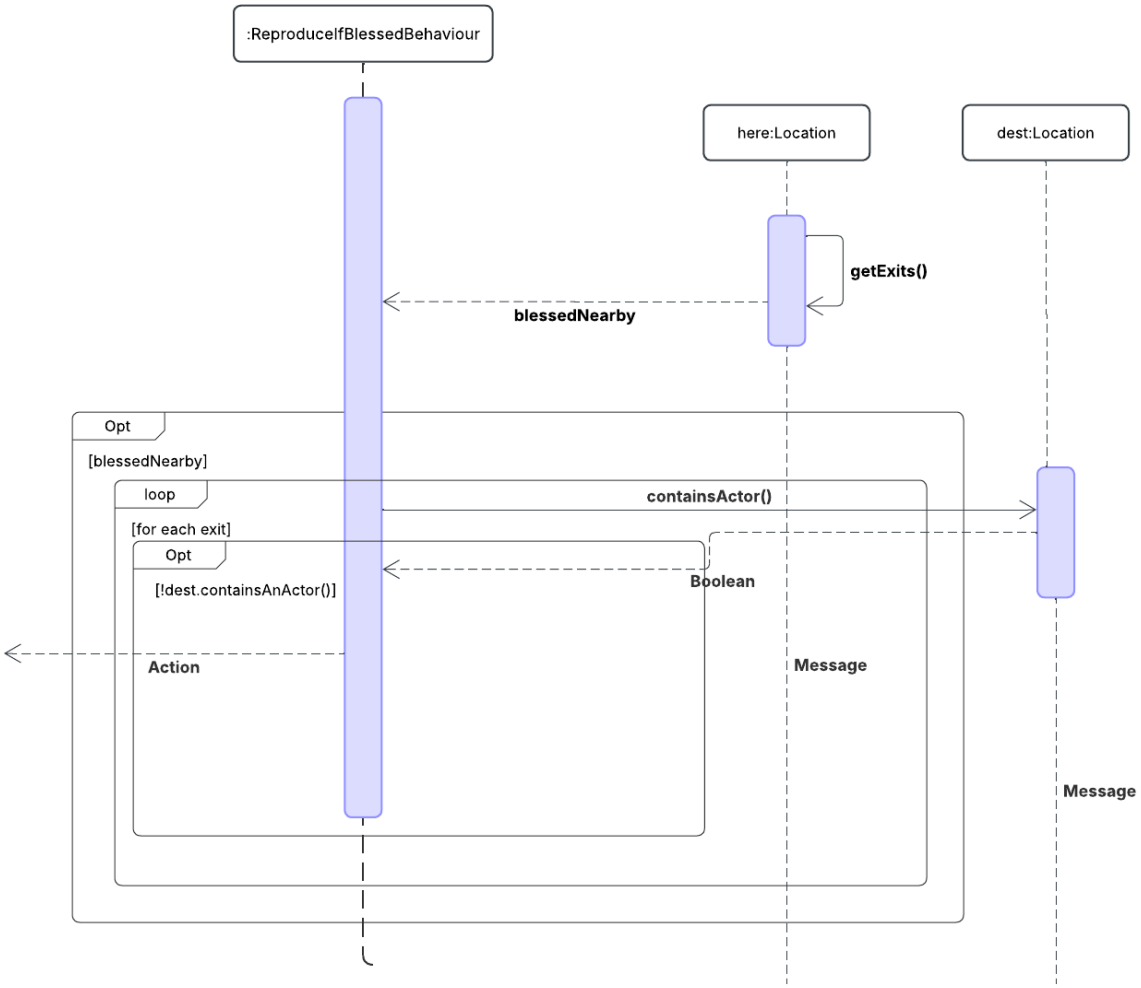
# REQ1

## Design - UML Diagram

Note: New classes are shown in purple



# Design - Sequence Diagram



## Design - Rationale

| Feature |
| --- |
| The BlessedUtils class offers a static isSurroundedByBlessed method to check if a location is adjacent to blessed entities or grounds. |
| **Justification** |
| Pros:<br>- Reusability<br>By defining this as a utility class, the isSurroundedByBlessed method can be reused across various game elements, such as SpiritGoat or other entities that need to check for a "blessed" condition.<br><br>- Encapsulation of Logic<br>The logic for detecting "blessed" surroundings is cleanly separated into a single class, reducing duplication and improving maintainability.<br>Cons:<br>- Limited Extensibility<br>Adding more contextual checks like different types of blessings may require modifying the utility class directly, violating the Open-Closed Principle. |
| **Alternative Design** |
| Another approach is to use a BlessingChecker interface with corresponding implementations. In this design, the SpiritGoat entity would implement the BlessingChecker interface to indicate whether they are blessed by grace. This promotes polymorphism and enhances extensibility, allowing new types of blessed entities to be easily added in the future without changing existing logic. However, this approach introduces additional complexity, as each relevant class must implement the interface, which can lead to scattered implementation details and increase the overall codebase maintenance effort. |

## Feature

The Capabilities enum defines attributes like BLESSED_BY_GRACE or CONSUMABLE_AS_EGG to represent behaviors of entities and grounds.

## Justification

Pros:

- Centralized Capability Management
  The enum serves as a single source of truth for all capabilities in the game, reducing redundancy.

- Readability
  The names of the capabilities are self-explanatory, making the code more readable.

Cons:

- Limited Extensibility
  Adding new capabilities requires modifying the Capabilities enum, which can lead to tight coupling and larger files.

## Alternative Design

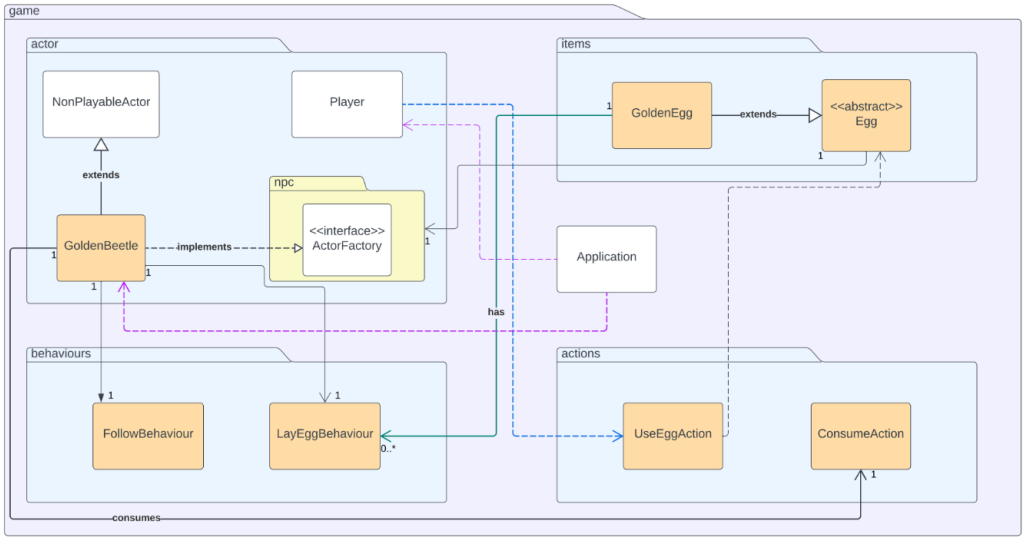
An alternative design is to use a Capability base class with subclasses representing specific capabilities, such as BlessedCapability or CurableCapability. This approach promotes extensibility by allowing each capability to encapsulate its own behavior, making the system more flexible and potentially more powerful. However, it also increases complexity by introducing multiple additional classes, which can make the codebase harder to navigate and maintain.

| **Feature** |
| --- |
| The Curable interface defines a cure method, enabling consistent behavior for curable entities. |
| **Justification** |
| Pros:<br>- Polymorphism<br>Different entities like OmenSheep and SpiritGoat can implement Curable in their own way, promoting flexibility.<br><br>- Extensibility<br>New curable entities can be added without modifying existing ones.<br>Cons:<br>- Single Responsibility Violation<br>If the cure logic grows complex, it may violate the single responsibility principle of the implementing classes. |
| **Alternative Design** |
| Using a Curable abstract class allows curable entities to inherit a default cure method implementation, reducing code duplication when common curing logic is shared, but it limits flexibility due to Java's single inheritance restriction. |

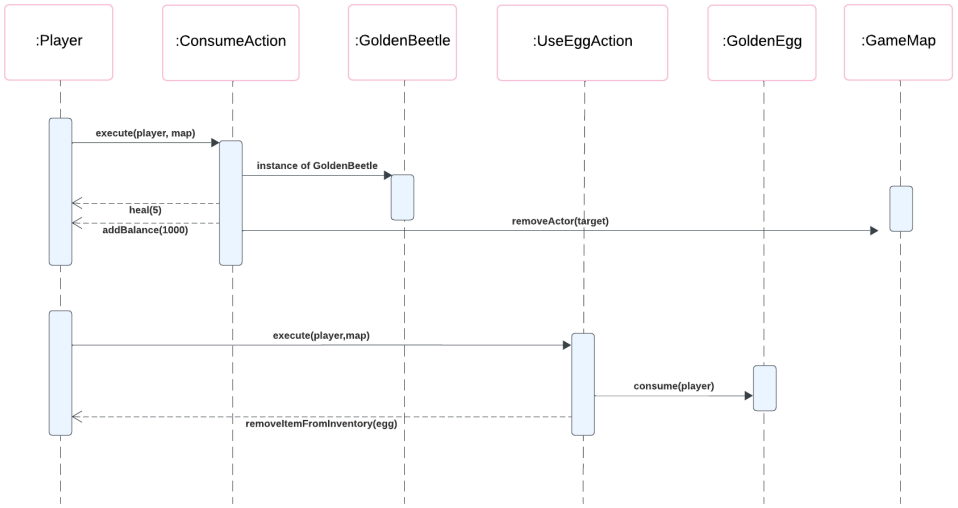
# REQ2

## Design - UML Diagram

Note: New classes shown in orange




## Design - Sequence Diagram



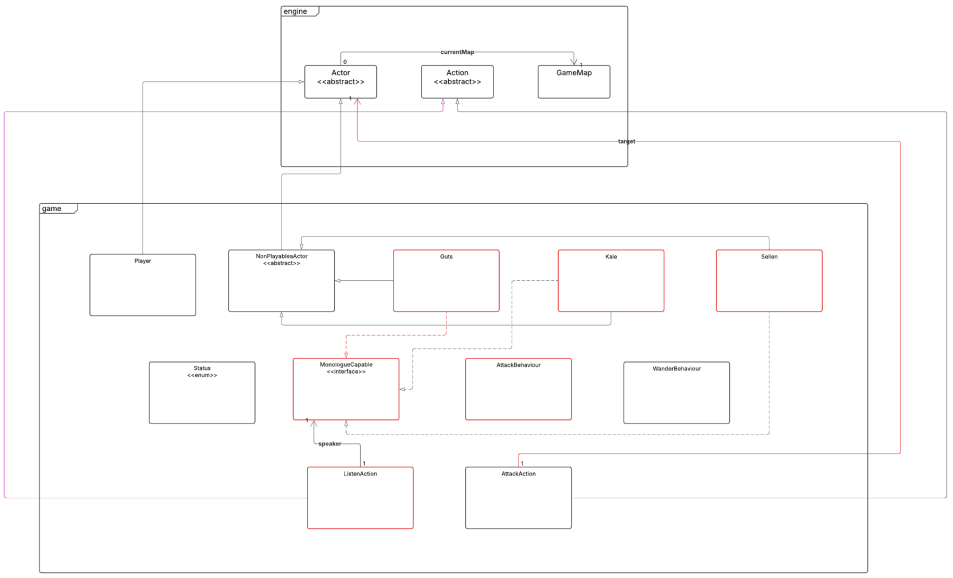
## Design - Rationale

| Feature |
| --- |
| Use of Egg Abstract Class with ActorFactory Pattern |
| **Justification** |
| Instead of handling egg hatching in the beetle or hardcoding the outcome, we created an Egg abstract class and used the ActorFactory interface to generate new beetles on hatch.<br><br>Pros:<br>- Future eggs can be added by extending Egg and defining hatch logic.<br>- Consumption and hatching are handled via overridden methods (consume(), hatchConditionMet()), removing the need for downcasting or conditionals.<br>- Hatching logic is handled within the egg's tick() method, reducing responsibility on map or actor classes.<br>- ActorFactory allows any Actor to be instantiated from eggs, supporting future requirements.<br>Cons:<br>- Requires new class and behaviour<br>- Each egg stores a reference to an actor factory (the parent beetle), which may be unnecessary if many egg types emerge. |
| **Alternate Design** |
| An alternative design was to handle egg-laying and hatching entirely within the GoldenBeetle class using a basic Item and hard coded logic. This was rejected as it would violate the Single Responsibility Principle, making the beetle responsible for too many tasks. It also lacked reusability and scalability for future creatures. The final design, using an abstract Egg class and the ActorFactory pattern, promotes modularity, extensibility, and cleaner separation of concerns. |

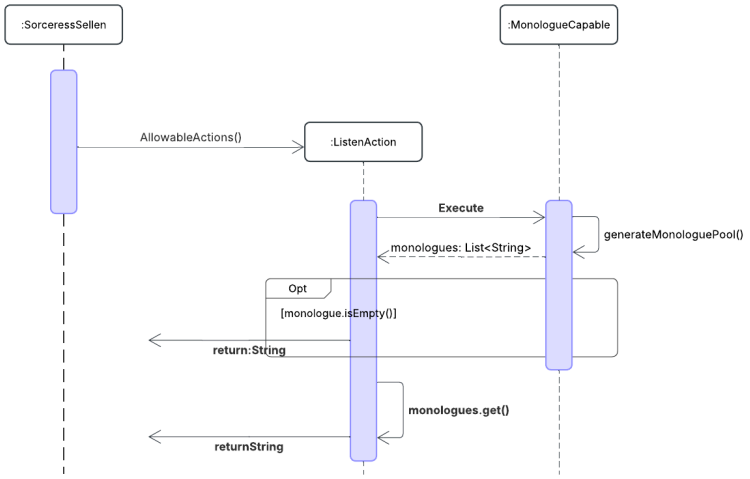
| Feature |
| --- |
| The Player consumes beetles directly on the map using ConsumeAction, and eggs from inventory using UseEggAction. |
| **Justification** |
| The design introduces two separate actions: ConsumeAction for consuming map-based actors like the Golden Beetle, and UseEggAction for consuming eggs from the Player's inventory. This separation ensures each action handles a specific interaction.<br><br>Pros:<br>- Clear separation of concerns between inventory and map-based consumption.<br>- Easily extendable for other consumable items or entities.<br>- Supports polymorphism via Egg.consume()<br>Cons:<br>- Slight code duplication between the two actions.<br>- Requires separate registration or handling logic in the player's turn. |
| **Alternate Design** |
| A single ConsumeAction could handle both map and inventory consumption by checking whether the target is an actor or an item. This was rejected as it introduces conditional branching, reduces clarity, and violates the Single Responsibility Principle. |

# REQ3

## Design - UML Diagram




## Design - Sequence Diagram



## Design - Rationale

| **Feature** |
| --- |
| The game introduces new NPCs (Sellen, Kale, Guts) who can speak contextual monologues when the player chooses to interact via ListenAction. Each of these NPCs implements the MonologueCapable interface to supply dynamic monologue options. Additionally, Guts is designed to attack any actor with HP > 50 using a new AttackBehaviour. |
| **Justification** |
| To support the requirement that certain NPCs provide dialogue based on specific conditions (player HP, nearby blight, wallet balance), we introduced the MonologueCapable interface. This allows each NPC to define their own logic in generateMonologuePool, returning a list of context-specific monologues. The ListenAction was introduced as an explicit interaction option that appears in the menu when a MonologueCapable NPC is adjacent to the player. This design avoids any need for downcasting or instanceof checks.<br><br>Three new NPCs were added:<br>● Sellen: A wandering NPC who always speaks from a fixed pool of two lines.<br>● Merchant Kale: A wandering NPC whose dialogue changes based on the player's rune balance, inventory, and blight.<br>● Guts: A hostile NPC with 500 HP, who attacks any actor with HP > 50. His monologue changes depending on the listener’s HP<br><br>To implement Guts’ aggression, a new AttackBehaviour was added. This behavior scans adjacent tiles and initiates an AttackAction on any target that satisfies the specified condition (HP > 50). AttackBehaviour is assigned a higher priority than WanderBehaviour, allowing Guts to attack nearby targets before wandering.<br><br>Pros:<br>● SRP: Dialogue logic is delegated to each NPC’s implementation of MonologueCapable, attack logic is handled by AttackBehaviour.<br>● OCP: New NPCs or monologue conditions can be added without modifying existing logic.<br>● LSP: MonologueCapable ensures any actor using ListenAction adheres to a predictable contract.<br>● ListenAction is reusable and decoupled from the specific logic of any NPC. |

- AttackBehaviour is flexible and may be reused for future aggressive NPCs.

Cons:

- Each NPC is responsible for writing their own condition-checking logic for monologues, which could lead to duplication.

- generateMonologuePool may grow complex for NPCs with many conditional cases (e.g Kale).

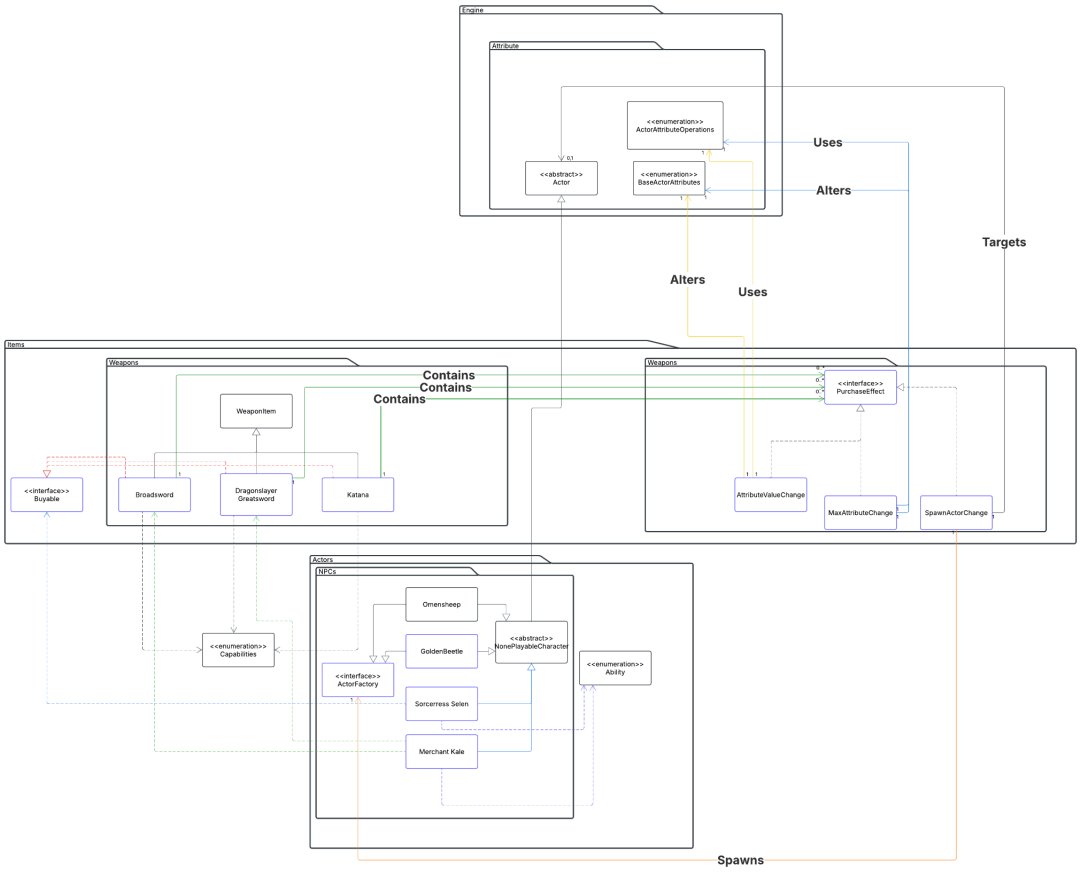
**Alternative Design**

An abstract class like MonologueNPC could have been used to consolidate shared monologue logic (checking for Blight or Runes), but this would limit inheritance flexibility and violate SRP by encouraging shared logic for characters with unrelated behaviors. A second option, embedding monologue data in a central MonologueManage was also avoided, as it would require centralized conditional logic and violate the open-closed principle.
Lastly, we could have triggered monologues automatically on proximity (rather than via ListenAction), but this would reduce player agency and clutter the UI with unsolicited messages and also goes against the requirement.

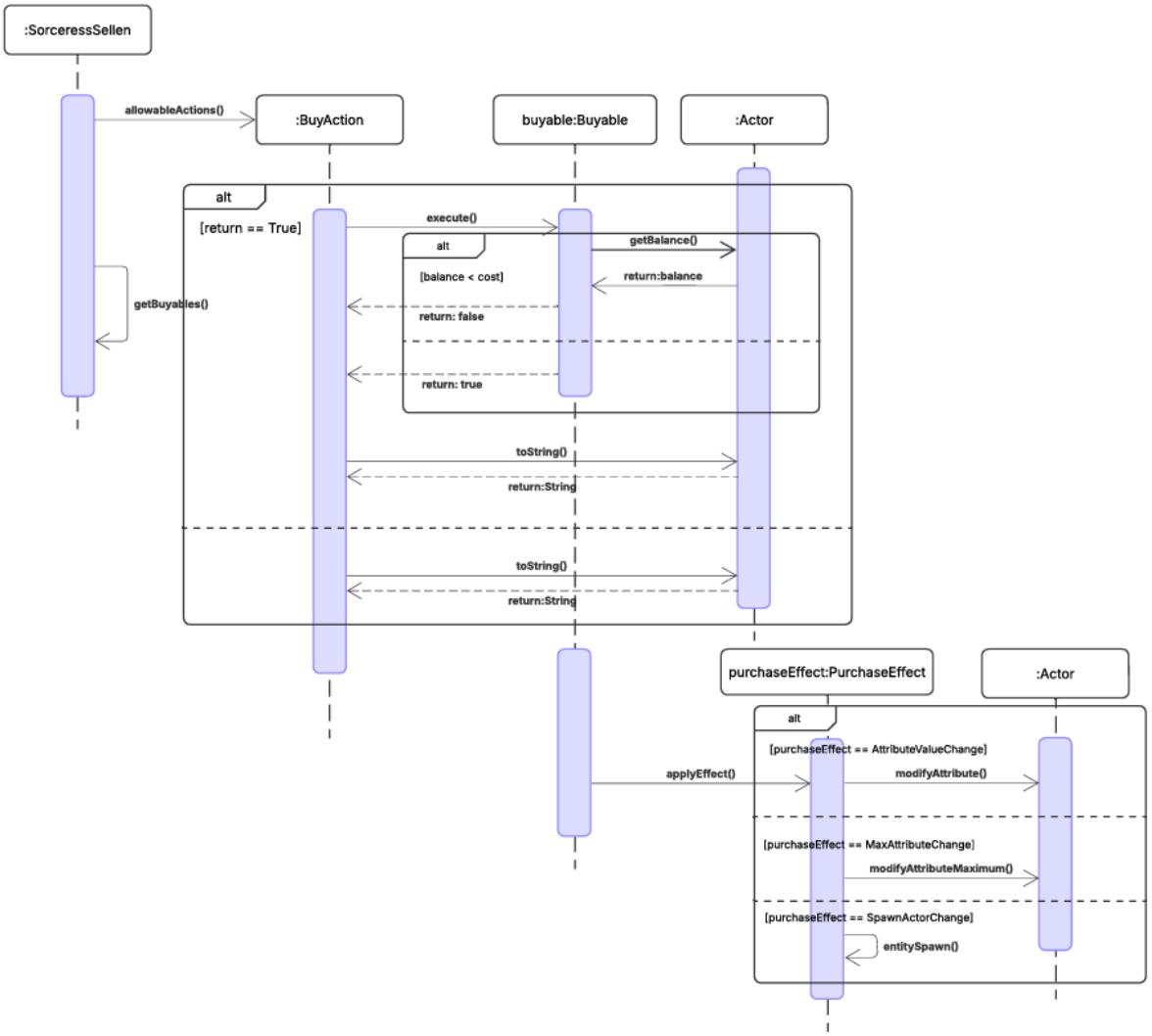
# REQ4

## Design - UML Diagram

Note: classes in blue are new classes



# Design - Sequence Diagram



## Design - Rationale

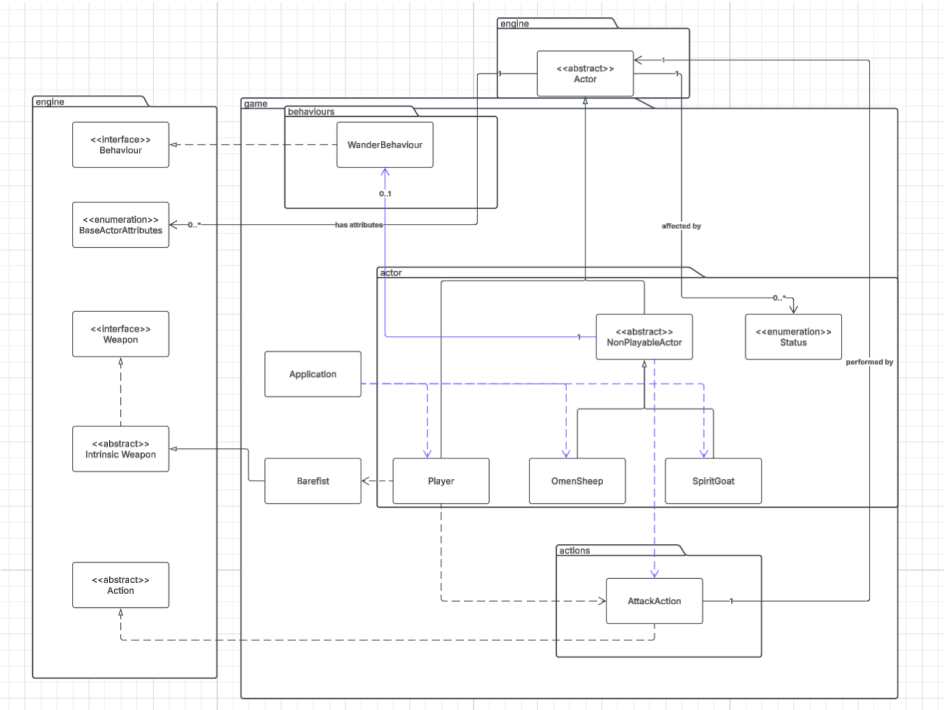
| **Feature** |
| --- |
| Weapons: Each weapon (Katana, Dragonslayer Greatsword, Broadsword) extends our WeaponItem concrete class |
| **Justification** |
| Each weapon has unique characteristics. To capture these characteristics, without having to define these effects every time each weapon is defined in every detail (such as name, char, damage, etc), a class was created for each weapon. As a result, one can simply create a new Katana instead of having to fill out the details of a new weapon item every time.<br>Pros:<br>- Reenforces SRP; WeaponItem is not responsible for features of every single weapon<br>- LSP is enforced as weapons share WeaponItem as parent.<br><br>Cons:<br>- Some minor deviations from DRY in the methods of each weapons (e.g. allowableActions)<br>- If cost or damage after purchase of Katana were to be updated, the class may need to be changed; which violates OCP |
| **Alternative Design** |
| All weapons could be instances of WeaponItem (not a child).<br>This would require some purchase effects (particularly one off ones) to be added to the object.<br>The reasons this was not chosen include deviations from SRP (as WeaponItem would need to be extended with new functions; increasing its responsibilities).<br>As mentioned above, one would also need to populate each weapon's characteristics each time (as opposed to just creating a new instance). |

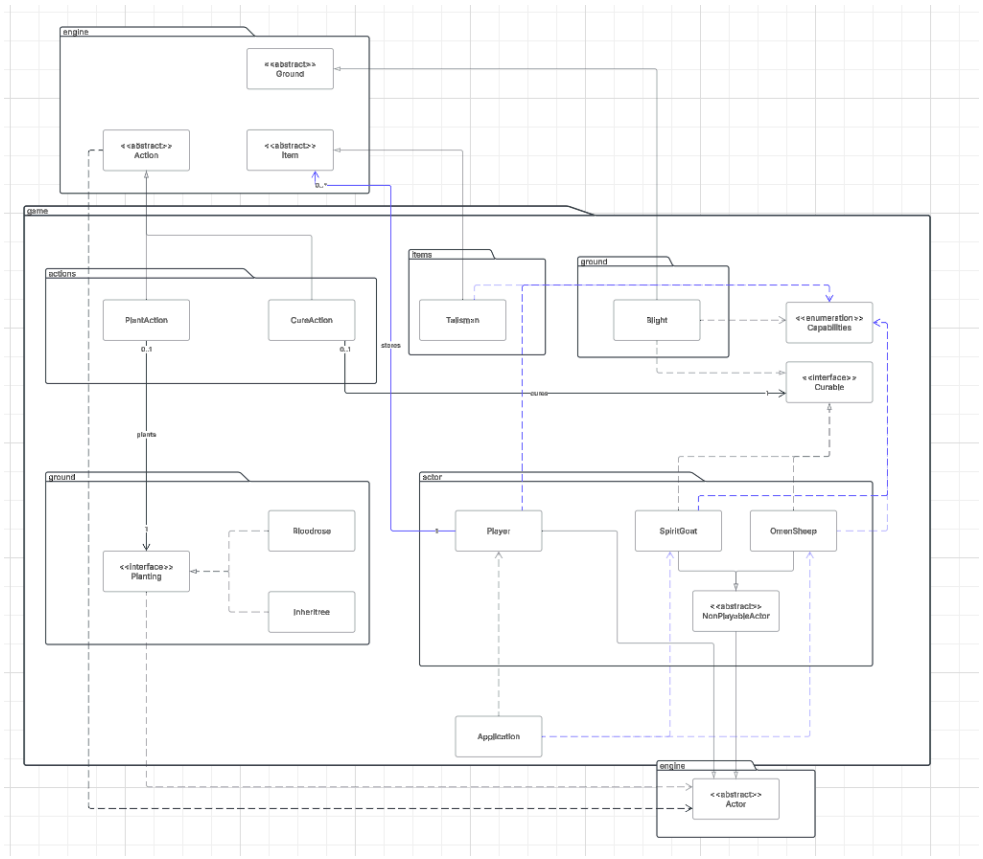
| **Feature** |
| --- |
| Purchases: Any Item that can be bought implements Buyable. BuyAction then takes in a Buyable to execute purchase |
| **Justification** |
| SRP was a key consideration in creating Buyable, as well as OCP. Buyable allows for new buyable items to be made with customs costs and purchase logic without having to change any existing code.<br><br>Pros:<br>- Simple interface<br>- Upholds ISP, as only buyable items implement buyable.<br>- Anything can be buyable (good for extensibility; for example if animal will be bought in future versions)<br><br>Cons:<br>- Since any Buyables can be sold, new features to all buyables may result in changing the interface. This is not an issue at this stage; trying to resolve this would result in overengineering.<br>- Each item's purchase logic has to be implemented separately. |
| **Alternative Design** |
| If all buyables were to have the same/similar buy logic, an abstract buyable class could be made with implementations to buy logic.<br>This would prevent repeated code. It would also allow for implementation or inclusion of more methods for use by ALL buyables.<br><br>This approach was avoided as REQ4 mentions that there may be weapons with different purchase effects. To enhance flexibility the interface approach was chosen. |

.

| **Feature** |
| --- |
| Purchase Effects: Most purchase effects are instances of PurchaseEffect. PurchaseEffects are passed into a given Weapons Items list of effects. Then executed upon purchase. |
| **Justification**<br><br>REQ4 clearly states “There could be other merchants selling the same items with new effects”. To make the code more extensive and to adhere to DIP, this ‘affects’ ’system was devised.<br>The main benefit is that at any time, the effects can be added and taken from an object, meaning that the purchase effects of items are fully customisable.<br>Another benefit is that effects can be baked into a weapon by default, further avoiding repetition and enhancing OCP.<br><br>SRP was a key consideration in creating Buyable, as well as OCP. Buyable allows for new buyable items to be made with customs costs and purchase logic without having to change any existing code.<br><br>SpawnEffect relies on ActorFactory to be implemented by the instance of Actor class. The reason for this choice was to allow any entity to be spawned.<br><br>Pros:<br>- Supports OCP: any effect can be added/removed without modifying the class<br>- DIP is adhered to; weapons don't rely on any single effect, they execute whatever effect is given to them<br>- Effects lists are full customisable at any time<br>-<br><br>Cons:<br>- If a NEW effect is to be added one either must implement a new instance of PurchaseEffects OR must hardcode this effect (which breaks OCP and must be avoided). |

# Previous UML Diagrams




engine
«abstract»
Ground
«abstract»
Action
«abstract»
Item
0..*
game
actions
PlantAction
CureAction
0..1
0..1
stores
items
Talisman
ground
Blight
«enumeration»
Capabilities
«interface»
Curable
cures
1
plants
ground
Bloodrose
«interface»
Planting
Inheritree
actor
Player
SpiritGoat
OmenSheep
«abstract»
NonPlayableActor
Application
engine
«abstract»
Actor